UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY



Introduction to Cryptography

Lecturer: Dr. Nguyen Minh Huong

E-VOTING SYSTEM

Group 5

Team members

Nguyen Quang Anh – 23BA14012
Hoang Thi Kieu Ly – 22BA13208
Pham Tuan Hai – 23BA14095
Ha The Duy – 2410207
Dinh Nhu Quynh – 23BA14246
Vu Tien Dat – 23BA14047
Phuong Anh Nam – 23BA14208

Hanoi, 12/05/2026

Table of Contents

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Purpose of the Project . . . 1

2 Cryptographic Algorithms **1**
2.1 RSA Blind Signature (Chaum, 1983) . . . 1
2.1.1 Mathematical Theory . . . 1
2.1.2 Application in System . . . 1
2.2 ElGamal Encryption (ElGamal, 1985) . . . 1
2.2.1 Mathematical Theory . . . 1
2.2.2 Application in System . . . 2
2.3 HMAC-SHA256 (Krawczyk, 1997) . . . 2
2.3.1 Mathematical Theory . . . 2
2.3.2 Application in System . . . 2
2.4 Baby-step Giant-step - BSGS (Shanks, 1971) . . . 2
2.4.1 Mathematical Theory . . . 2
2.4.2 Application in System . . . 2

3 Mechanism of Operation **2**
3.1 The Cryptographic Synergy . . . 2
3.2 System Workflow Phases . . . 2

4 Testing & Performance Analysis **4**
4.1 Test Cases . . . 4
4.2 Stress Test (Locust) . . . 4
4.3 Performance Benchmarks . . . 5

5 Conclusion & Future Work **5**
5.1 What the System Contributes . . . 5
5.2 Open Question: Would this system scale to a national election? . . . 5
5.3 Future Directions . . . 5

References **6**

1 Introduction

1.1 Motivation

Digital elections are convenient, but ensuring every vote counts exactly once, without revealing who cast it, is far harder than it sounds. Transparency and anonymity cannot coexist easily.

1.2 Purpose of the Project

This project set out to answer one practical question: "Can a university election be made both anonymous and verifiable using nothing but standard cryptographic primitives?"

To achieve this, the team built a custom crypto module combining RSA Blind Signature, ElGamal Homomorphic Encryption, and HMAC-SHA256 that runs entirely in the voter's browser, ensuring a ballot is encrypted before it ever leaves the device. The backend, built on Python, Flask, and SQLite, enforces a strict one-vote-per-student rule while making it mathematically impossible, even for the database administrator, to link any ballot back to its owner. The frontend, a lightweight single-page interface in Tailwind CSS and vanilla JavaScript, keeps the entire cryptographic flow invisible to the voter.

2 Cryptographic Algorithms

2.1 RSA Blind Signature (Chaum, 1983)

RSA is an asymmetric scheme based on the integer factorization problem. A blind signature allows the authority to sign a token without knowing its content, ensuring voter anonymity.

2.1.1 Mathematical Theory

The algorithm uses a blinding factor r where $\gcd(r, n) = 1$:

- Blind: $m' = m \cdot r^e \pmod{n}$
- Sign: $s' = (m')^d \pmod{n}$
- Unblind: $s = s' \cdot r^{-1} \pmod{n}$
- Verify: $s^e \equiv (m \cdot r^e)^d \cdot r^{-1 \cdot e} \equiv m^d \pmod{n}$

2.1.2 Application in System

This is used for Identity Separation. The Election Authority signs a blinded token to verify a student's eligibility. The student then unblinds it to obtain a valid credential. Since the authority never sees the original token, the vote remains completely anonymous.

2.2 ElGamal Encryption (ElGamal, 1985)

Based on the Discrete Logarithm Problem. This system utilizes Exponential ElGamal to convert multiplicative homomorphism into additive homomorphism, which is essential for summing votes.

2.2.1 Mathematical Theory

Key generation uses a prime p and generator g. Public key is (p, g, β) where $\beta = g^d \pmod{p}$:

- Standard Encryption: $c_1 = g^i, c_2 = M \cdot \beta^i \pmod{p}$. This supports multiplicative homomorphism.
- Exponential ElGamal: By encoding vote M as g^M before encryption, we get $c_2 = g^M \cdot \beta^i \pmod{p}$.
- Additive Homomorphism: Multiplying two ciphertexts results in the sum of the exponents:

$$\prod Enc(g^{M_i}) = (\prod g^{i_n}, \prod g^{M_n} \cdot \beta^{i_n}) = Enc(g^{\sum M_i}) = Enc(g^T)$$

- Decryption: $k_M = c_1^d \rightarrow g^T = c_2 \cdot k_M^{-1} \pmod{p}$.

2.2.2 Application in System

Guarantees Client-Side Data Security. Ballots are encrypted as g^M in the browser — the additive homomorphic property aggregates all votes into $Enc(g^T)$ without viewing any individual ballot, with T recovered via BSGS.

2.3 HMAC-SHA256 (Krawczyk, 1997)

A mechanism for message authentication using a cryptographic hash function combined with a secret key.

2.3.1 Mathematical Theory

The function uses a secret key K and inner/outer padding:

- Formula: $HMAC(K, m) = H((K \oplus opad) \parallel H((K \oplus ipad) \parallel m))$
- SHA-256: Operates on 512-bit blocks to produce a 256-bit digest.

2.3.2 Application in System

Used for System Integrity and Pre-Filtering. Every packet is sealed with an HMAC, allowing the server to instantly drop tampered or unauthorized packets, defending against ballot-flooding and DDoS attacks.

2.4 Baby-step Giant-step - BSGS (Shanks, 1971)

A time-memory tradeoff algorithm designed to solve the discrete logarithm problem $\beta = g^x$ efficiently.

2.4.1 Mathematical Theory

Rewrite $x = x_g \cdot m + x_b$ where $m = \lceil \sqrt{|G|} \rceil$:

- Baby-step: Compute and store g^{x_b} for $0 \leq x_b < m$ in a hash table.
- Giant-step: For each x_g, check if $\beta \cdot (g^{-m})^{x_g}$ matches any entry in the table.
- Complexity: Reduces computation from $O(N)$ to $O(\sqrt{N})$.

2.4.2 Application in System

Used for Blind Tallying. After the server decrypts the aggregate ciphertext, it receives the value g^T. BSGS is then used to recover the integer T (total votes) from g^T quickly and accurately.

3 Mechanism of Operation

3.1 The Cryptographic Synergy

A secure election needs three things: anonymity so no one knows who voted for whom, privacy so no one can read the ballot, and integrity so no one can fake or change a vote. No single algorithm can do all three. RSA Blind Signature hides the voter's identity, but it does not encrypt the ballot. ElGamal encrypts the ballot, but it cannot detect if someone altered the packet on the way to the server. HMAC catches any tampering, but it does not hide anything. Together, each one covers exactly what the others miss. The four phases below show how they work as a single system.

3.2 System Workflow Phases

Note: To maintain a high-level focus on system architecture and ensure narrative continuity, detailed implementation scripts are moved to the Appendix and the source code repository[1]*, including* `app.py` *and the* `crypto/` *module.*

[1] https://github.com/MRXz194/E-voting

System Flow Diagram:

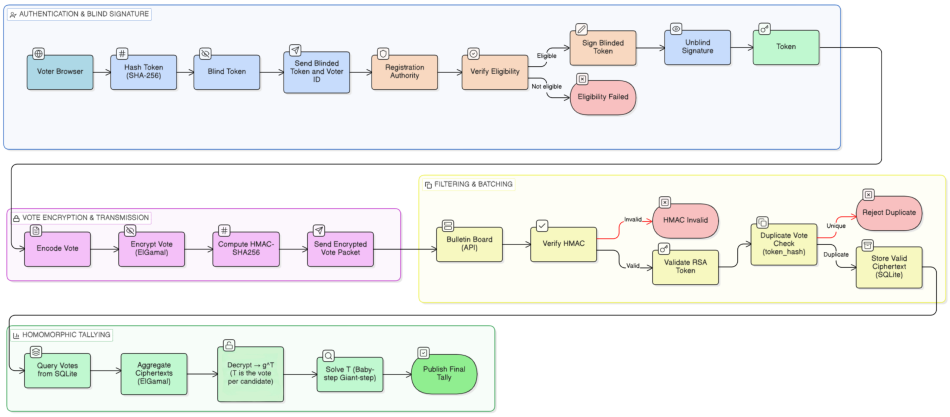


Figure 1: Overall System Architecture and Transaction Flow

Phase 1 — Authentication & Blind Signature (RSA)

Before casting a ballot, the system must confirm a voter is eligible without learning who they are. To do this, the browser generates a random token, hashes it, and masks it with a blinding factor r before sending it to the Registration Authority (RA):

$$m' \equiv H(\text{Token}) \cdot r^e \pmod{n}$$

The RA signs this hidden value without ever seeing the actual token content. After receiving the signature, the voter's browser removes the blinding factor r to obtain a clean credential (s). This credential is mathematically valid but completely decoupled from the voter's identity:

$$s' \equiv (m')^d \pmod{n} \implies s \equiv s' \cdot r^{-1} \pmod{n}$$

Phase 2 — Vote Encryption & Transmission (ElGamal + HMAC)

To ensure privacy, the browser encrypts the vote using Exponential ElGamal. A random nonce k is used for every ballot, ensuring that even identical votes result in drastically different ciphertexts:

$$c_1 \equiv g^k \pmod{p}, \quad c_2 \equiv g^M \cdot y^k \pmod{p}$$

However, encryption alone cannot prevent an attacker from modifying the data while it travels across the network. To block this, the browser creates an HMAC-SHA256 "seal" using a session key K. This seal binds the encrypted vote to the voter's credential. If anyone tampers with the packet, the server will instantly detect the mismatch:

$$K = \text{SHA256}(\text{Token} \parallel \text{Credential})$$
$$\text{HMAC} = \text{HMAC-SHA256}(K,\ \text{Token} \parallel c_1 \parallel c_2)$$

Phase 3 — Filtering & Batching

When a ballot reaches the server, it must pass through a strict three-layer security pipeline:

- Integrity Check: The server verifies the HMAC seal. If broken, the request is dropped (HTTP 400), preventing forged packets from consuming verification resources.
- Eligibility Check: The system verifies the RSA credential by checking $s^e \equiv H(\text{Token}) \pmod{n}$, proving a genuine RA signature while keeping the voter's identity hidden.
- Uniqueness Check: The final layer enforces the "one-vote-per-token" rule by checking the token hash against the database; duplicates trigger a 409 rejection.

Outcome: Only ballots navigating all three layers are stored as valid ciphertexts in the database.

Phase 4 — Homomorphic Tallying (ElGamal + BSGS)

After the election closes, the server retrieves all stored ballots. Using the homomorphic property of ElGamal, it multiplies the ciphertexts for each candidate together. This creates a single, combined encrypted total without ever decrypting individual votes:

$$C_1 \equiv \prod c_{1,i} \pmod{p}, \quad C_2 \equiv \prod c_{2,i} \pmod{p}$$

The server decrypts this combined total just once to obtain the blended value g^T. Finally, it uses the Baby-step Giant-step (BSGS) algorithm to solve the discrete logarithm problem and recover T, the final vote count for that candidate:

$$g^T \equiv C_2 \cdot C_1^{-d} \pmod{p} \implies T = \text{BSGS}(g, g^T, p)$$

This process ensures a transparent and verifiable result while maintaining 100% voter privacy.

4 Testing & Performance Analysis

4.1 Test Cases

To evaluate correctness and limits, we built a test tool to attack every part of our security design.

Test Case (Python Module)	Description	Expected-System Response	Time	Result
Group 1: Correctness (End-to-End Flow)				
`test_full_happy_path_flow`	Register → Vote → Receive receipt	64-char receipt returned	2.24s	✓PASS
Group 2: Double-Vote Resistance & Replay Attacks				
`test_double_voting_1000_attempts_all_stacked`	Send 1000 votes with the same token	Only 1 stored, 999 rejected (409)	1.90s	✓PASS
`test_credential_cannot_be_reused_with_fake_token`	Use a valid credential with a fake token	Rejected (HTTP 403)	0.18s	✓PASS
Group 3: HMAC & RSA Integrity (Tampering Forgery)				
`test_tampered_ciphertexts_rejected_by_hmac`	Modify c_1 after signing, keep old HMAC	HMAC mismatch, dropped (400)	0.39s	✓PASS
`test_forged_credential_is_rejected`	Use a random number as credential	RSA verify fails (HTTP 403)	3.88s	✓PASS
Group 4: Known Vulnerability (Critical Thinking)				
`test_zkp_vulnerability_attack`	Encode 1000 votes for Alice in 1 ballot	System blindly accepts it	3.39s	[!] GAP

Table 1: Automated Security Audit & Test Results (Executed via pytest in 12.70s)

ZKP Limitation Analysis: Test 6 highlights a constraint in basic Exponential ElGamal. Without Zero-Knowledge Proofs (ZKP) to verify data ranges, an attacker could encrypt an invalid weight (e.g., g^{1000}) instead of a standard 0 or 1. This vulnerability remains a documented priority for future security enhancements.

4.2 Stress Test (Locust)

We tested the system using Locust to identify its performance limits under concurrent traffic.

Scenario	Users	Ramp Up	Duration	Failure Rate	Avg Response
• Smoke Test	10	1 user/s	1 min	0%	∼35 ms
• Load Test	100	10 users/s	3 min	20%	∼200 ms

Table 2: Performance Metrics under Simulated Voting Traffic

Note: While the application logic is secure, the use of SQLite limits concurrent voting due to its single-writer lock mechanism. Future versions will migrate to PostgreSQL to handle higher traffic volumes.

4.3 Performance Benchmarks

To quantify optimization efficiency, we compared our cryptographic methods against baseline approaches.

Module	N	Baseline	Optimized	Improvement
Tallying	100	32.57 ms	0.45 ms	98.6%
(ElGamal)	1,000	257.91 ms	2.06 ms	99.2%
	5,000	1,065.57 ms	5.58 ms	99.5%
Discrete Log	100	0.08 ms	3.58 ms	-6.1%
(BSGS)	1,000	0.23 ms	0.14 ms	69.6%
	5,000	1.05 ms	0.10 ms	89.2%
Filtering	100	0.50 s	0.00 s	99.93%
(HMAC)	1,000	5.28 s	0.00 s	99.93%
	5,000	26.16 s	0.02 s	99.91%

Table 3: Scaling Performance Gains Across Cryptographic Modules

Data shows that homomorphic tallying maintains $O(1)$ decryption time regardless of N. While BSGS has an initial overhead at $N = 100$, it significantly outperforms brute force as the system scales ($O(\sqrt{N})$). This overhead arises because building the hash table at small N costs more than simply iterating through all candidates. HMAC filtering consistently eliminates nearly 100% of invalid workload.

Overall, the benchmarks confirm that security and efficiency are both achieved at scale.

Note: A video demo of the system UI is available here.

5 Conclusion & Future Work

5.1 What the System Contributes

This project demonstrates that a fully anonymous, verifiable e-voting system can be built using only standard cryptographic primitives, without relying on third-party crypto libraries or trusted third parties. The combination of RSA Blind Signature (anonymity), ElGamal Homomorphic Encryption (privacy), and HMAC-SHA256 (integrity) provides a mathematically rigorous guarantee of:

- Anonymity: No link between voter identity and ballot
- Privacy: Ballots encrypted client-side, never decrypted individually
- Integrity: Tampered packets rejected before reaching crypto layer
- Verifiability: Voters can verify their receipt on the public Bulletin Board

5.2 Open Question: Would this system scale to a national election?

The current architecture uses SQLite (single-writer lock) and 256-bit ElGamal keys. For national scale (millions of voters), the system would require:

- PostgreSQL or distributed database for concurrent writes
- Larger key sizes (2048+ bits) for long-term security
- Distributed tallying nodes for parallel homomorphic aggregation

5.3 Future Directions

- Zero-Knowledge Proof (ZKP): Add range proofs to close the ballot manipulation gap found in TC A-19
- Blockchain Ledger: Replace the central Bulletin Board with a tamper-proof distributed ledger
- Threshold Decryption: Split the private key so no single authority holds full decryption power

References

[1] William Stallings. *Cryptography and Network Security: Principles and Practice*. Pearson, 7th edition, 2017.

[2] Christof Paar and Jan Pelzl. *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer Science & Business Media, 2009.

[3] David Chaum. Blind signatures for untraceable payments. In *Advances in Cryptology: Proceedings of CRYPTO '82*, pages 199–203. Plenum, 1983.

[4] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.

[5] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. HMAC: Keyed-Hashing for Message Authentication. Technical Report RFC 2104, Internet Engineering Task Force (IETF), 1997.

[6] Daniel Shanks. Class number, a theory of factorization, and genera. In *Proceedings of Symposia in Pure Mathematics*, volume 20, pages 415–440. American Mathematical Society, 1971.

[7] R. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21:120–126, 1978.

[8] R. Cramer, R. Gennaro, and Berry Schoenmakers. A secure and optimally efficient multi-authority election scheme. *European Transactions on Telecommunications*, pages 103–118, 1997.

[9] Ke Yuan, Peng Sang, Suya Zhang, Xi Chen, Wei Yang, and Chunfu Jia. An electronic voting scheme based on homomorphic encryption and decentralization. *PeerJ Computer Science*, 9, 2023.

[10] R. Cramer, I. Damgård, and Berry Schoenmakers. Proofs of Partial Knowledge and Simplified Design of Witness Hiding Protocols. In *Advances in Cryptology — CRYPTO '94*, pages 174–187. Springer, 1994.

[11] R. Cramer, M. Franklin, Berry Schoenmakers, and M. Yung. Multi-Authority Secret-Ballot Elections with Linear Work. In *Advances in Cryptology — EUROCRYPT '96*, pages 72–83. Springer, 1996.

[12] A. Singh and P. Chandran. A Secure and Efficient Multi-authority Proactive Election Scheme. In *Information Systems Security (ICISS 2007)*, pages 208–218. Springer, 2007.

[13] Feilong Hu. Research Based on Fully Homomorphic Encrypted Electronic Voting Scheme. *Advances in Applied Mathematics*, 2022.

[14] Hyunyeon Kim, Kyung Eun Kim, Soohan Park, and Jongsoo Sohn. E-Voting System Using Homomorphic Encryption and Blockchain Technology to Encrypt Voter Data. *arXiv preprint arXiv:2111.05096*, 2021.

[15] Group 5. E-Voting System: Source Code Repository. https://github.com/MRXz194/E-voting.

[16] Group 5. E-Voting System: Interactive UI Demo. https://www.youtube.com/watch?v=QvJzgJXNShc.